`

Computer Architecture Introduction

Veena S
Department of CSE
AHEAD CS
Amrita Viswa Vidyapeetham

Objectives

- Computer Architecture.
- CPU basics.
- Stored Program Concepts.

Computer Architecture

- Rules for defining the functionalities of a computer.
- Describing the capabilities and programming model of a computer.
  - Instruction set architecture
  - Micro Architecture
  - Logic Design

CPU Basics

Three parts

- ALU
- Control Unit
- Registers
  - Special Purpose Register
  - General Purpose Register

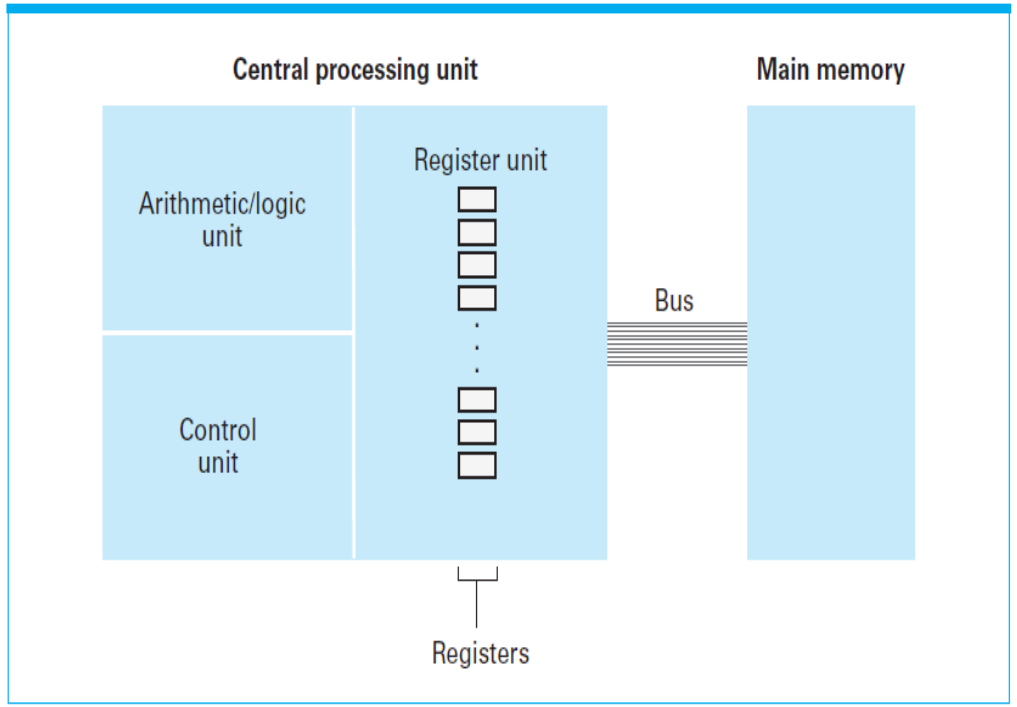


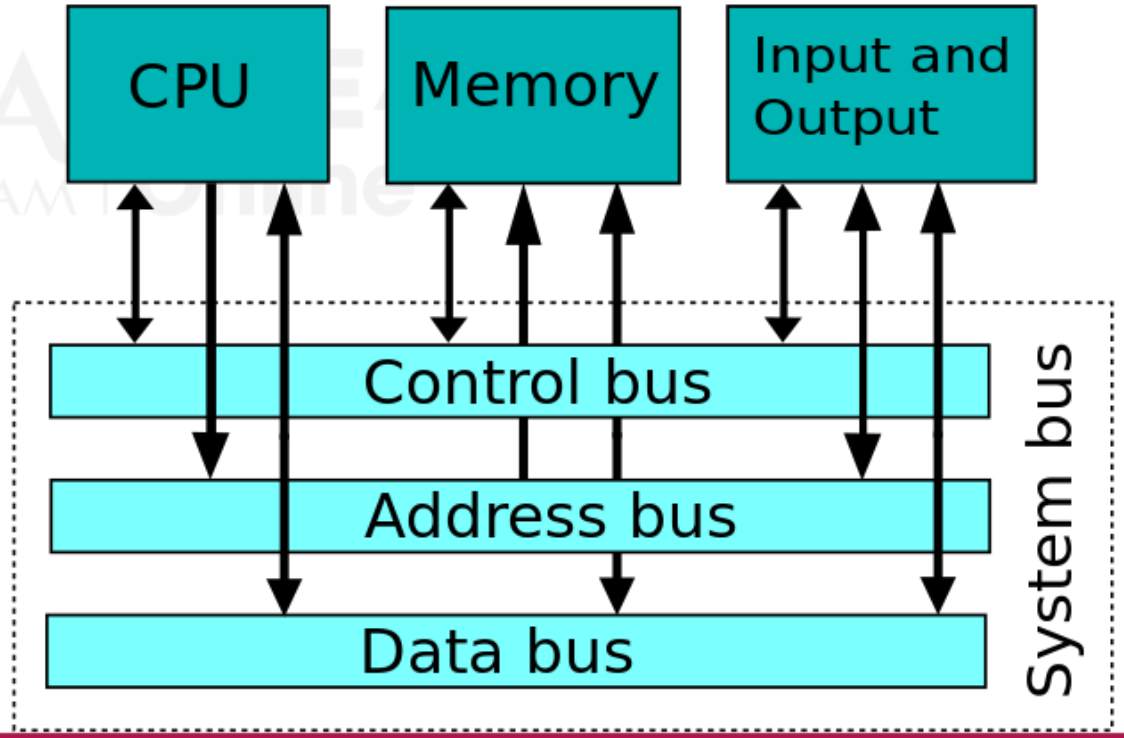
Image courtesy: J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelfth Edition

Registers

- Fast storage locations that hold data temporarily.
- Memory Address Register(MAR): Holds addresses.
- Memory Data Register(MDR): Holds the data.
- Instruction Register(IR): Holds the instruction currently being executed.
- Program Counter(PC): Points to the next instruction that is to be fetched from memory.

Bus Structure

- Group of wires which carries information between CPU and peripherals.
  - Data Bus
  - Address Bus
  - Control Bus.



CPU Basics

- CPU and main memory connected through wires called **bus.**
- Memory Read

  CPU places the address of the required cell in the bus along with an electronic signal.
- Memory Write

  CPU places address of the cell and the data in the bus along with a write signal.

Stored Program Concept

- Storing programs along with data in memory.
- Programs encoded as bit patterns and stored in memory.
- Control unit decode and execute the instructions.
- Programs can be updated in the memory.

Steps for Multiplying two Number

- Perform binary multiplication with values store in memory

1. Get the first value from the memory and store it in one of the registers.
2. Get the second value from the memory and store it in another register.
3. Activate the ALU to perform multiplication with the two registers as input and store the result in another register.
4. Store the result in memory.
5. Stop.

Machine Language

- CPU should recognize the instructions encoded as bit patterns.
- Collection of instructions along with encoded system is called machine language.
- Set of all instructions recognized by a machine.
- In machine language instructions are machine level instructions.

Instruction Set Architecture(ISA)

- Two CPU architectures.
- Reduced Instruction Set Computer(RISC)
  - Minimal set of machine instructions.
  - Such machines are efficient, fast, and less expensive to manufacture
- Complex Instruction Set Computer(CISC)
  - Large number of complex instructions.
  - More complex CPU can better cope with the ever increasing complexities of todays' software.

Instruction Format

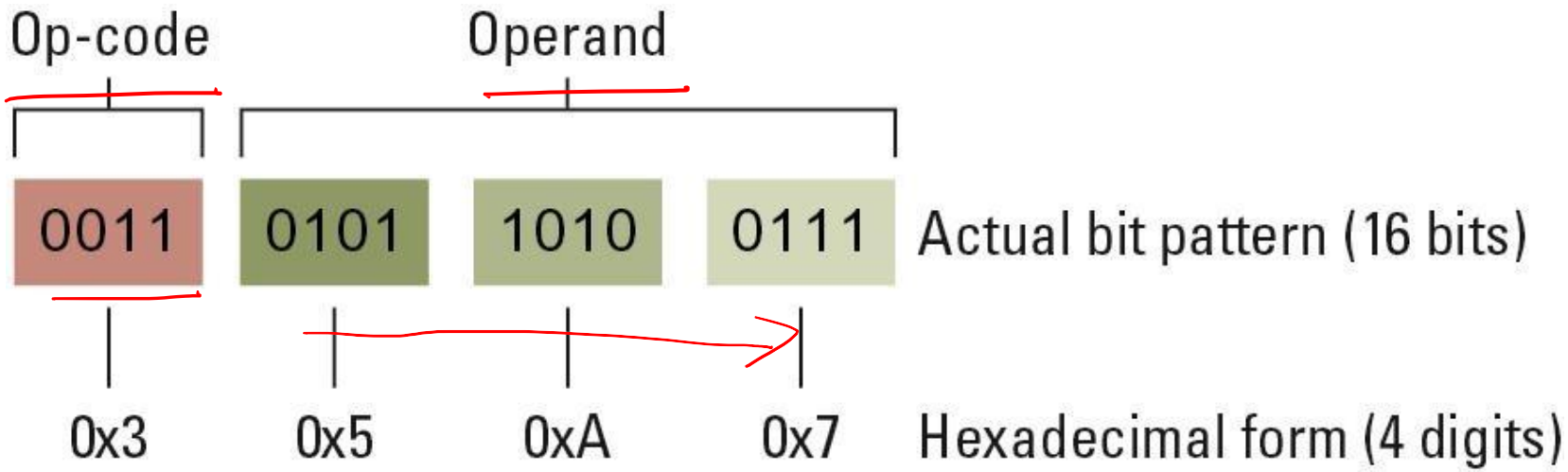


Image courtesy: J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelfth Edition

- Opcode- Specifies operation executed by the instruction.
- Operand- Data / memory location executing the instruction.

Instruction Formats

- Three address Instruction.
  - Operation Source1 Source 2 Destination
- Two address Instruction
  - Operation Source Destination
- One address Instruction
  - Operation Source/Destination
- Zero address Instruction
  - Location of all operands are defined implicitly

Instruction Types

- Data Transfer instructions.
- Arithmetic/logic instructions.
- Control instructions.

Instruction Types

- Data Transfer instructions.

  Transfer of information from one location in the computer to another.
- Register Transfer Notation.
- LOAD: Request to fill a general purpose register with the content of memory.
- STORE: Transfer the content of the register to memory cell.

Instruction Types

Arithmetic and Logic Instructions.

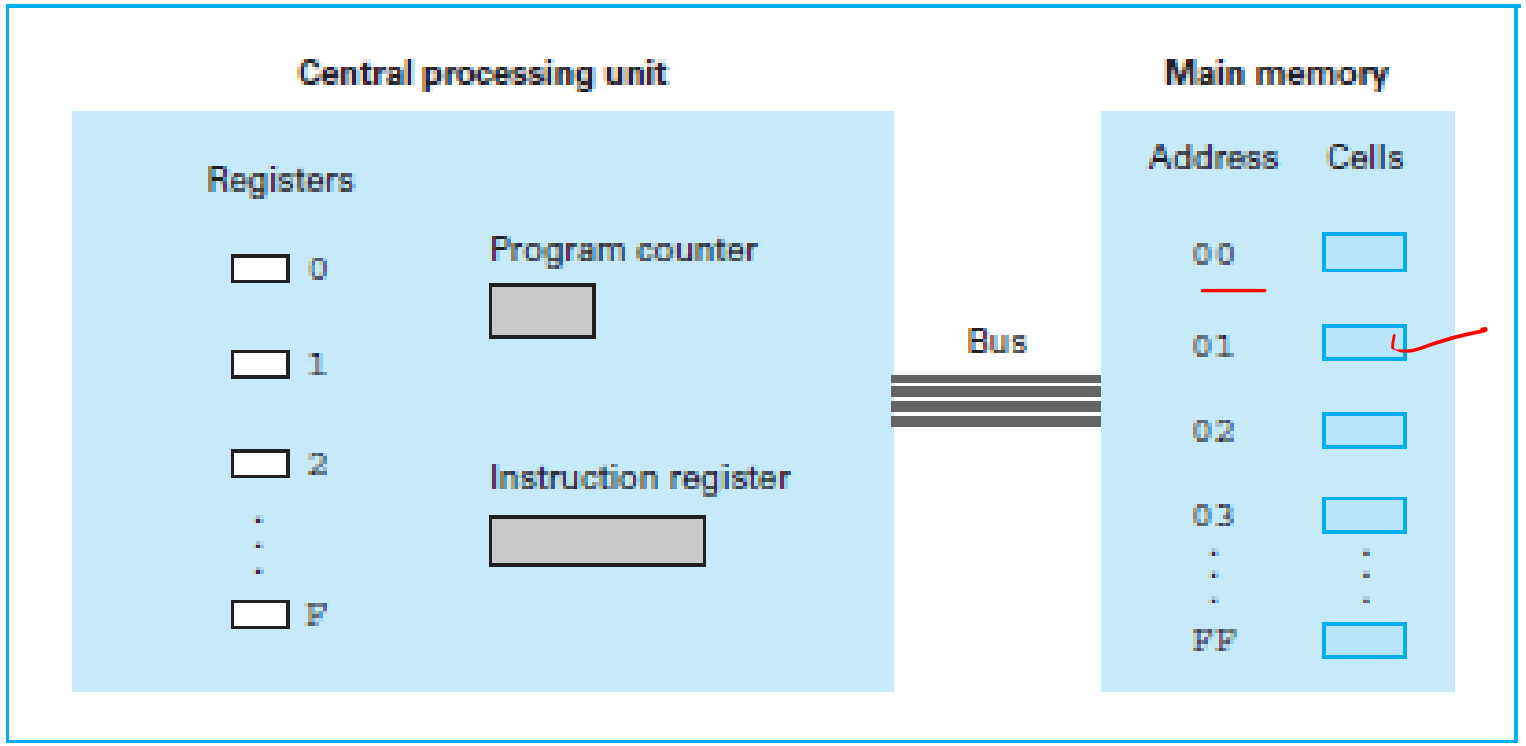
- Instructions that direct the ALU unit to perform some arithmetic or logic operations.
- ADD, SUB, AND, OR ,etc..
- Shift and rotate operations.

Instruction Types Example

DIV A,B,C

1. LOAD a register with a value from memory.
2. Load the second register with a memory content.
3. If the second value is zero JUMP to step6.
4. Perform division operation and divide the contents of the first register by second register and store the result in a third register.
5. Store the contents of the third register in memory.
6. Stop.

Encoding of Instructions



Machine Architecture

Image courtesy: J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelfth Edition

Instruction Encoding

- Bit pattern in the opcode field specifies which operation is requested by the instruction.
- Bit pattern at the operand field give more information about the operation.

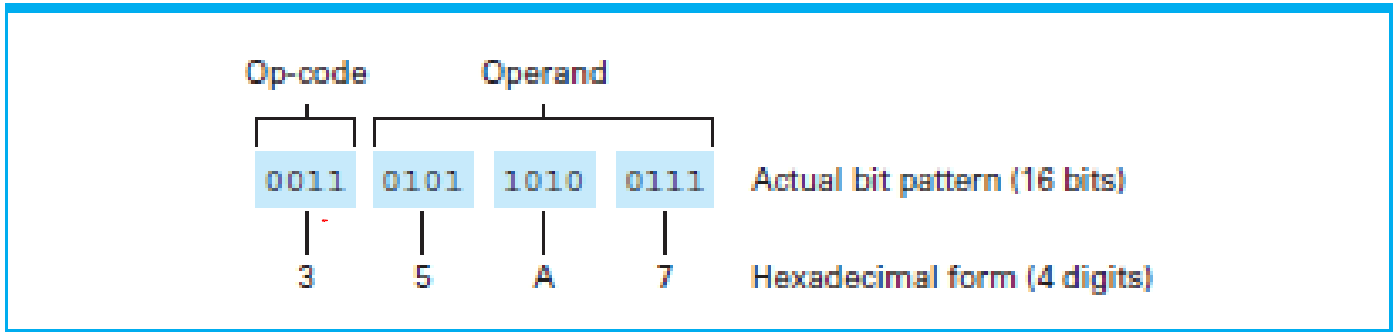
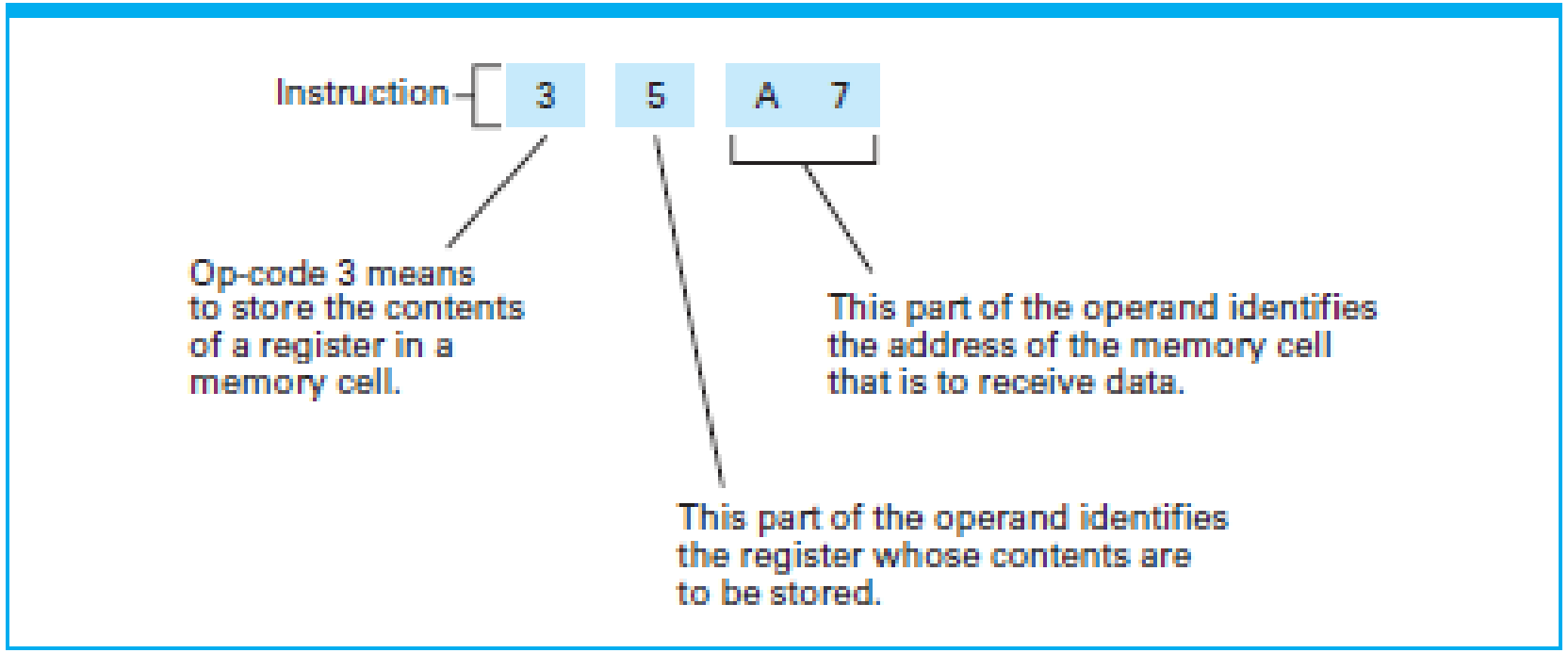


Image courtesy: J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelfth Edition

Instruction Decoding



Program Execution

- Fetch the instruction from the memory.
- Decode the instructions.
- Execute the instruction.

- Instruction Register(IR): Holding the instruction that is being executed.
- Program Counter(PC): Hold the address of the next instruction to be executed.

Machine Cycle

- CPU perform program execution by machine cycle
- Three step process.
- Fetch- Decode- Execute.

Machine Cycle

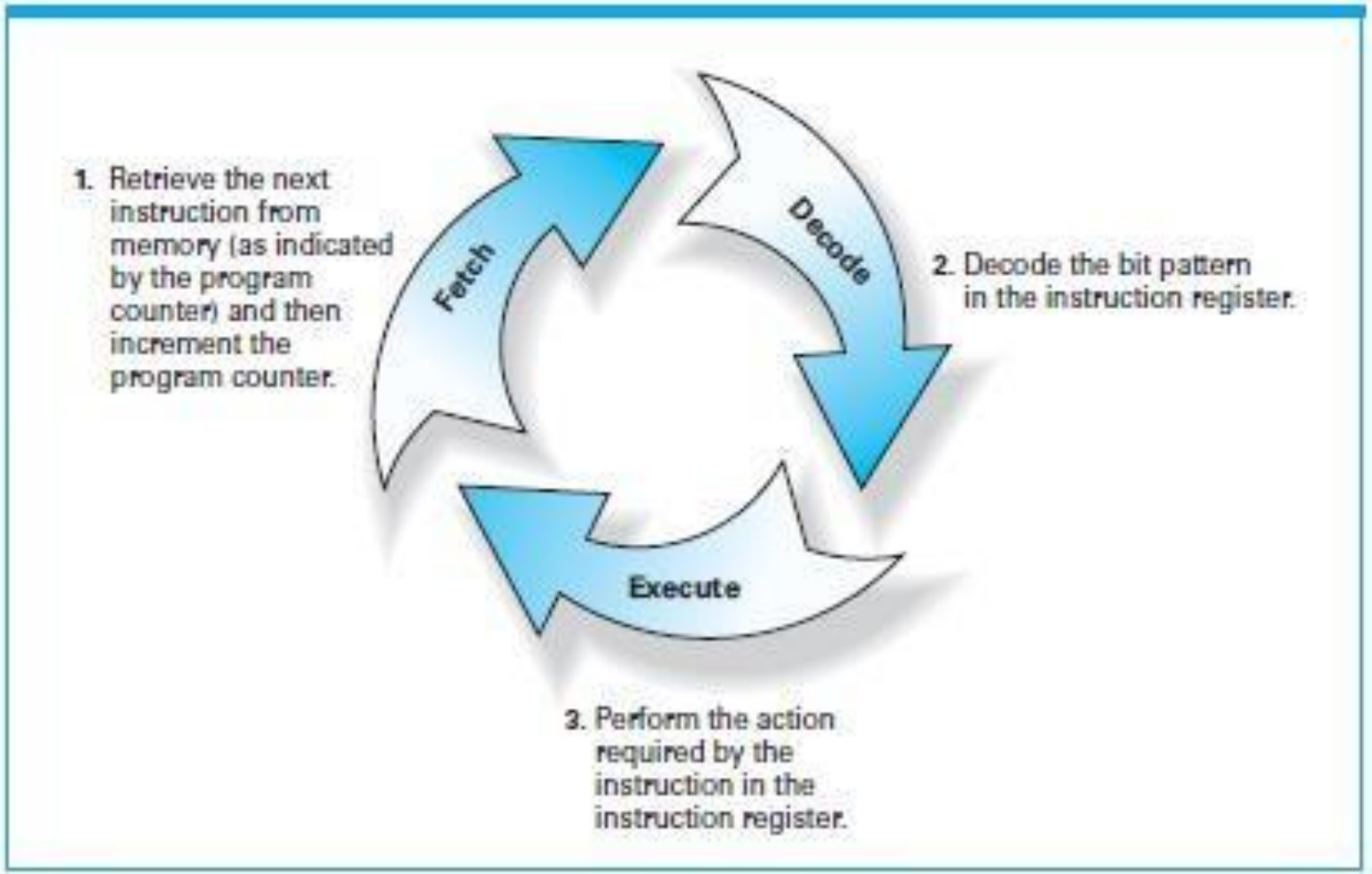


Image courtesy: J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelfth Edition

Example Program

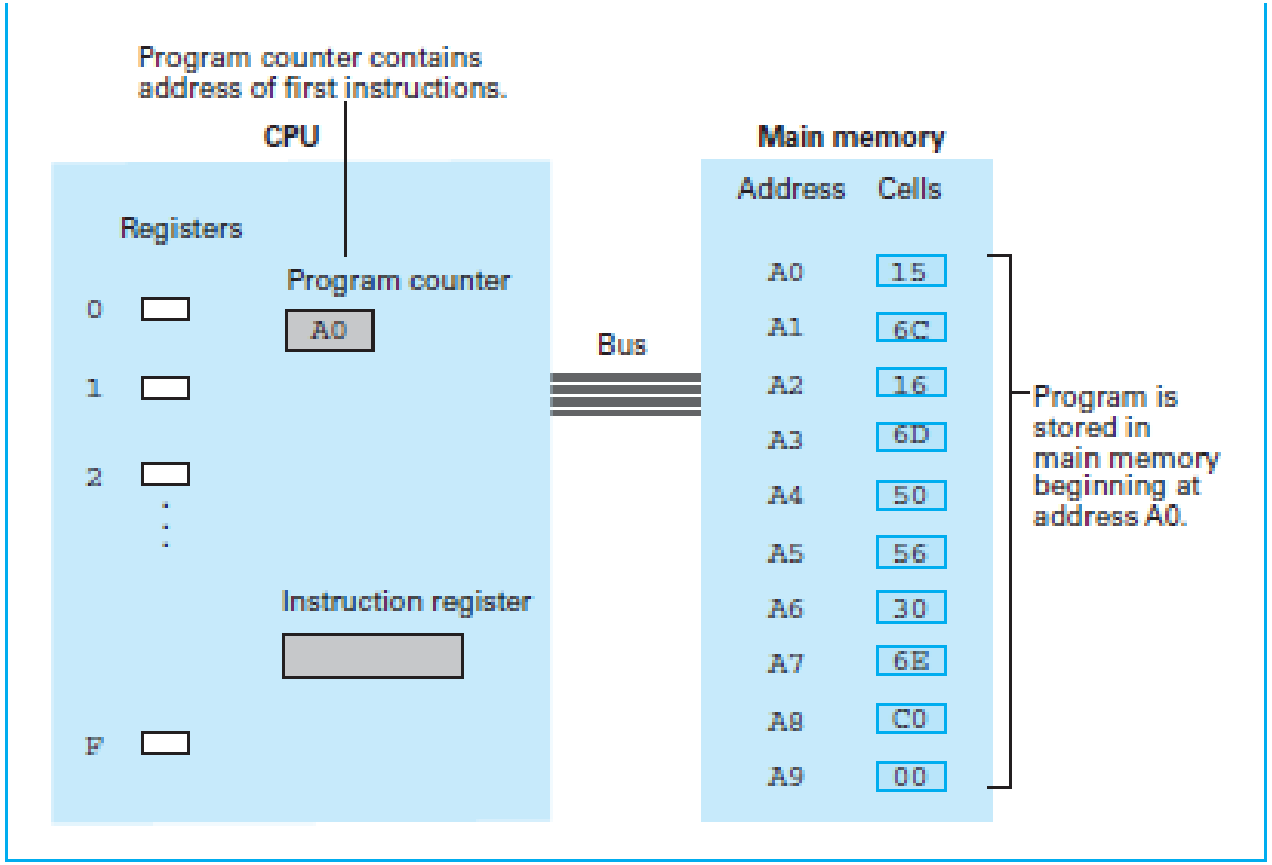
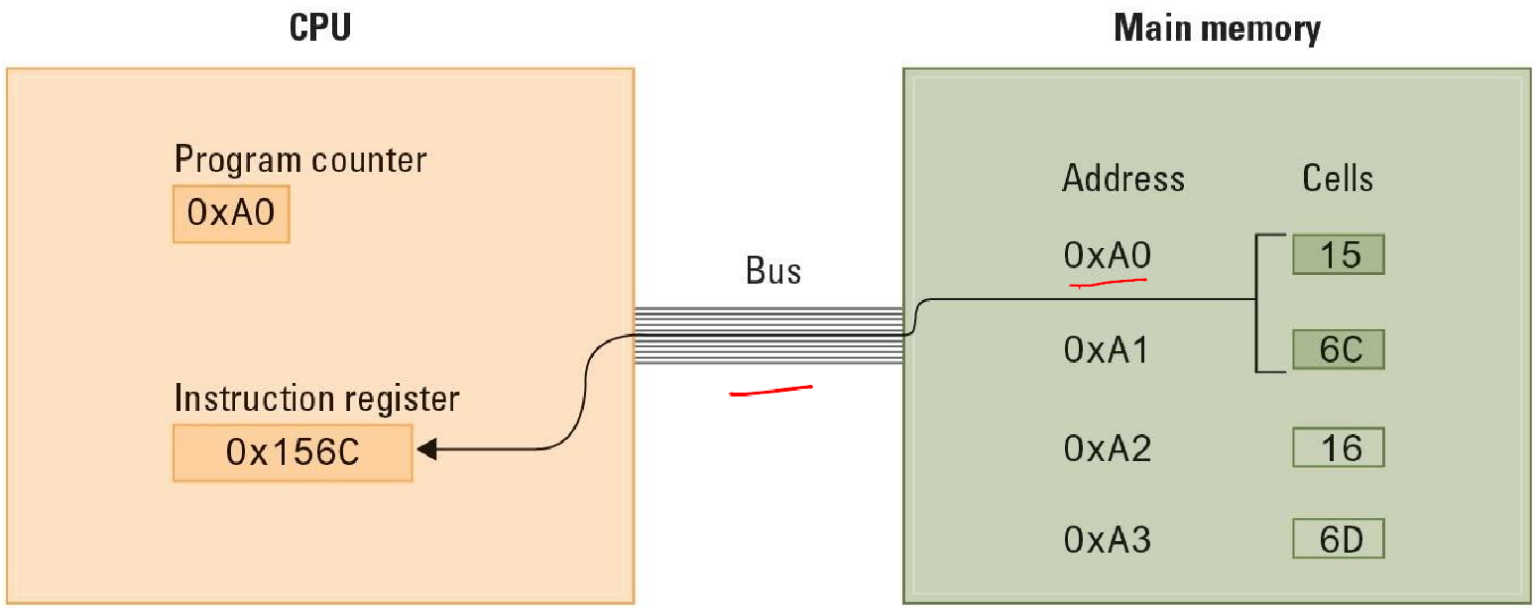


Image courtesy: J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelfth Edition

Instruction Fetch

- The content of the program counter has been taken.
- Corresponding memory cells content copied to the instruction register.
- Update the program counter.

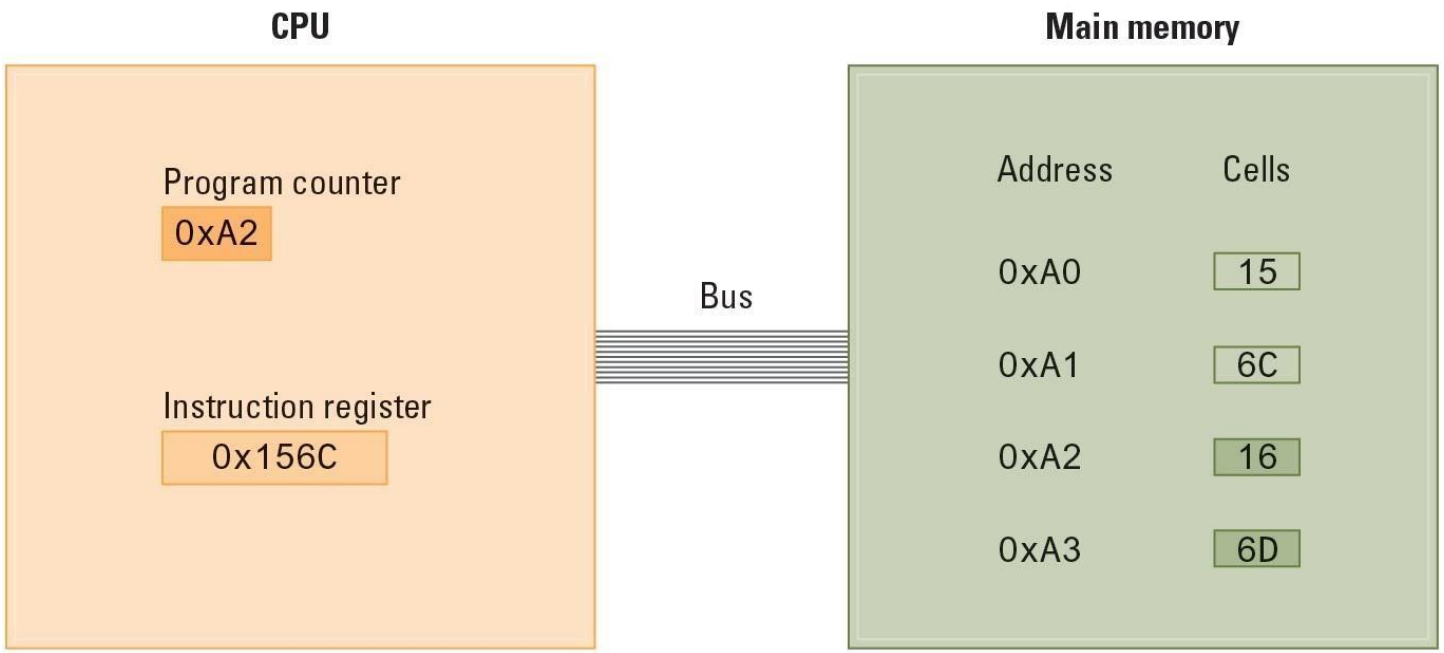
Instruction Fetch



a. At the beginning of the fetch step, the instruction starting at address 0xA0 is retrieved from memory and placed in the instruction register.

Image courtesy: J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelfth Edition

Instruction Fetch



b. Then the program counter is incremented so that it points to the next instruction.

Image courtesy: J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelfth Edition

Instruction Decode

- Decode the instruction in IR.
- Instruction decoder will then generate the signals to activate the circuitry to carry out the instruction.

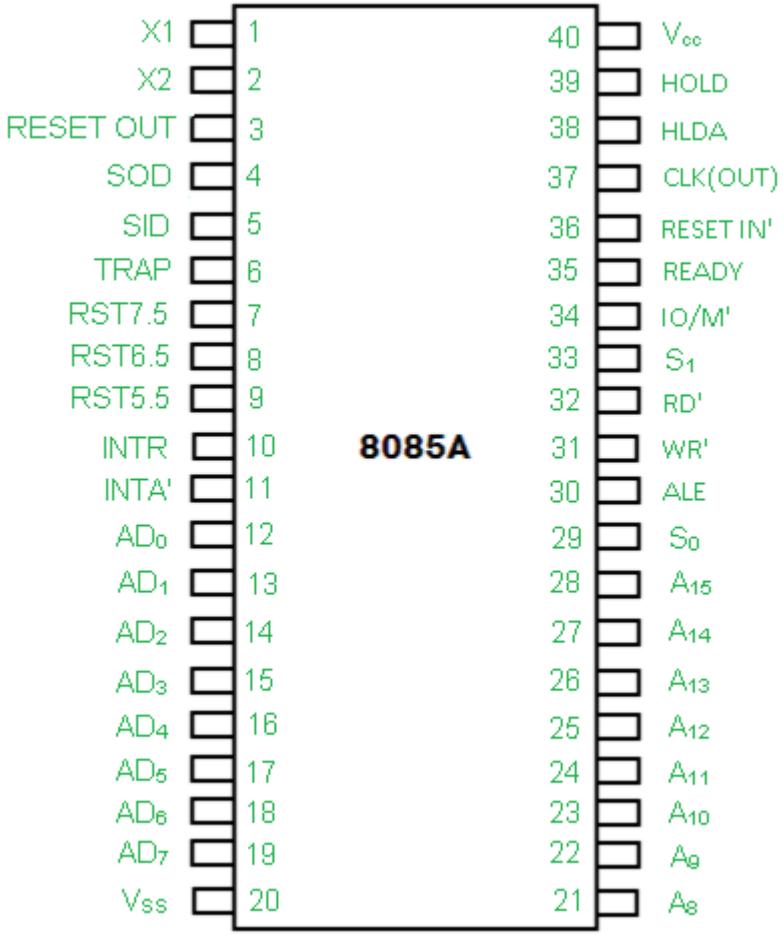
Instruction Decoding and Execution

- Instruction register contain 156C(Encoded form)
- CPU decode it.
- 156C- Load register 5 with the content of the memory cell 6C.
- Perform load on execution step.
- Start the next machine cycle.

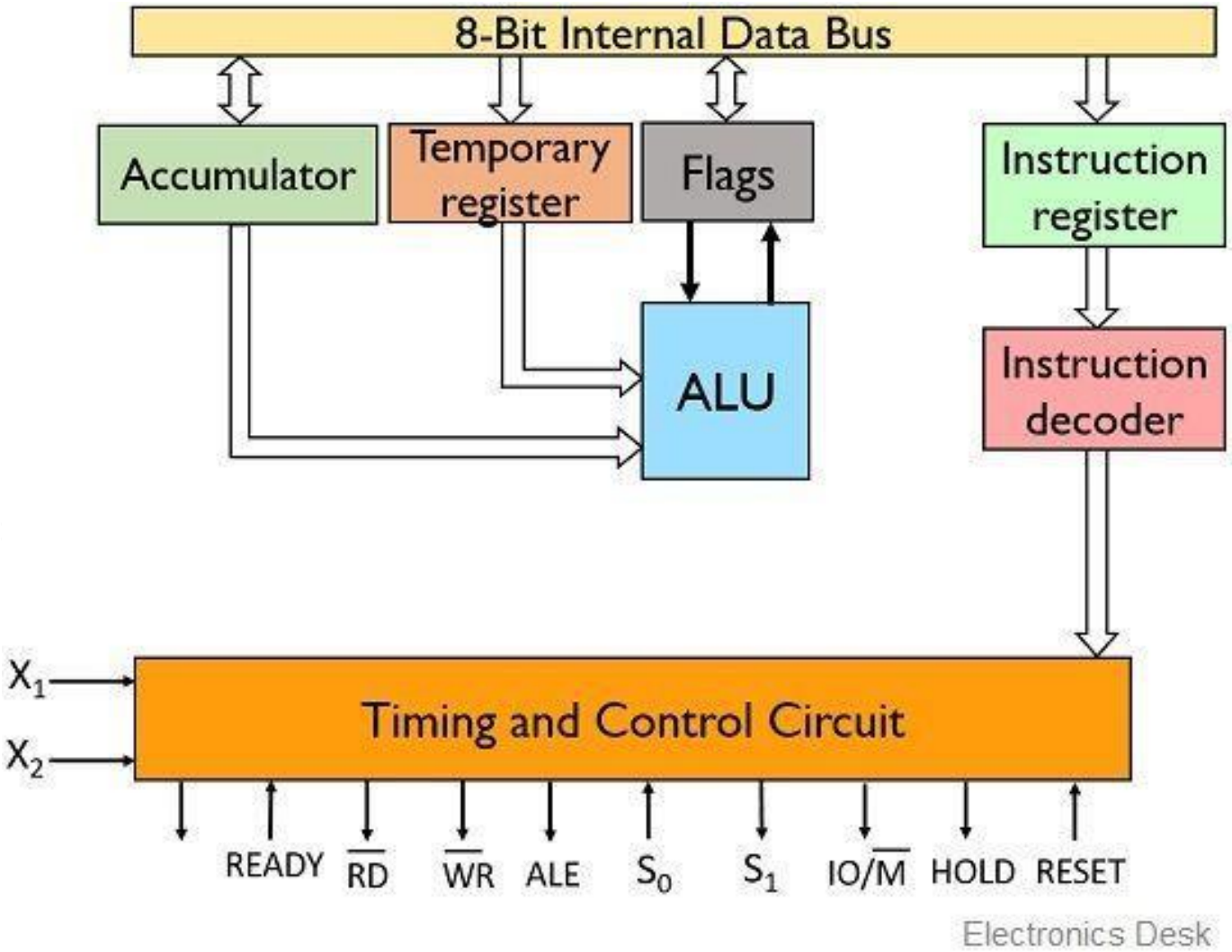
Objectives

- Basic Architecture of 8085 Microprocessor

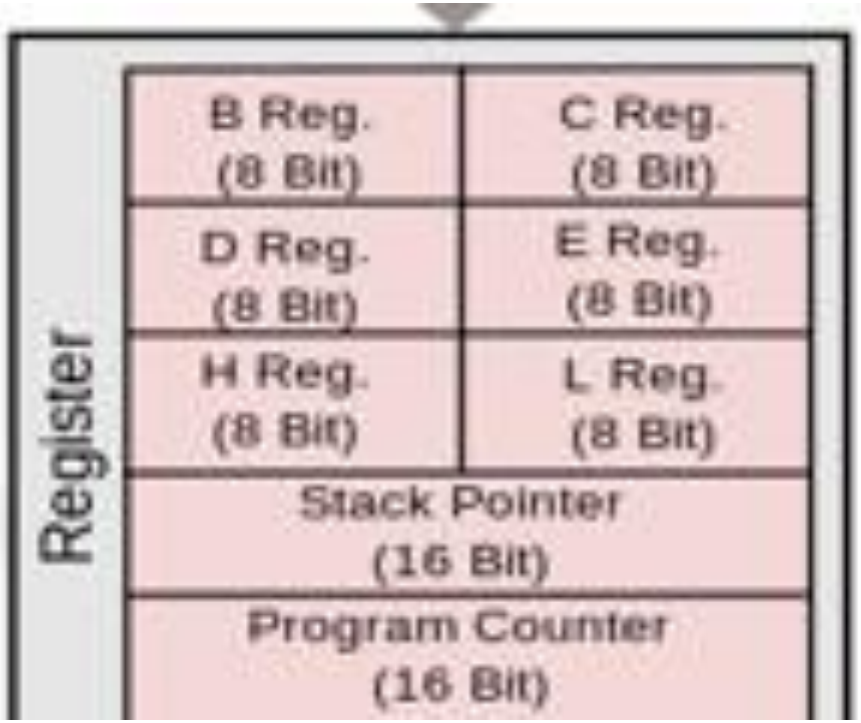
The 8085 MPU Internal Structure



The 8085 MPU Internal Structure



The 8085 MPU Internal Structure



S	Z	----	AC	---	P	---	CY

The 8085 MPU Internal Structure

The internal architecture of the 8085 MPU is capable of performing the following operations:

- Store 8-bit data (Registers, Accumulator)
- Perform arithmetic and logic operations (ALU)
- Test for conditions (IF / THEN)
- Sequence the execution of instructions
- Store temporary data in RAM during execution

The 8085 Bus Structure

- The 8-bit 8085 CPU (or MPU – Micro Processing Unit) communicates with the other units using a bus.
  - Address bus : 16bit.
  - Data Bus and Control bus: 8 bit.

The 8085 Bus Structure

Address Bus

- Consists of 16 address lines: $A_0 - A_{15}$
- Operates in unidirectional mode: The address bits are always sent from the MPU to peripheral devices, not reverse.
- 16 address lines are capable of addressing a total of $2^{16} = 65{,}536$ (64k) memory locations.
- Address locations: 0000 (hex) – FFFF (hex)

The 8085 Bus Structure

Data Bus

- Consists of 8 data lines: $D_0 - D_7$
- Operates in bidirectional mode.
- Data range: 00 (hex) – FF (hex)

Control Bus

- Consists of various lines carrying the control signals such as read / write enable, flag bits.

Registers

Registers

- Six general purpose 8-bit registers: B, C, D, E, H, L
- They can also be combined as register pairs to perform 16-bit operations: BC, DE, HL
- Registers are programmable (data load, move, etc.)

Accumulator

- Single 8-bit register that is part of the ALU !
- Used for arithmetic / logic operations –the result is always stored in the accumulator.

Registers

Flag Bits

- Indicate the result of condition tests.
- Carry, Zero, Sign, Parity, etc.
- Conditional operations (IF / THEN) are executed based on the condition of these flag bits.

Stack Pointer (SP)

- Contains the memory address (16 bits) of the top most memory location in the stack.

Program Counter (PC)

- Contains the memory address (16 bits) of the instruction that will be executed in the next step.

Takeaways

- 8085 Microprocessor architecture.

Intel 8085 ALU

The ALU performs the following arithmetic and logical operations.

- Addition
- Subtraction
- Increment (add 1)
- Decrement (subtract 1)
- Logical AND
- Logical OR
- Logical EXCLUSIVE OR
- Complement (logical NOT)
- Left shift

8085 Instruction Set

8085 Microprocessor has 74 instructions in its instruction set.

These 74 instructions are categorized into following 5 based on their operations.

1. Data Transfer Instructions
2. Arithmetic Instructions
3. Logical Instructions
4. Branching Instructions
5. Machine Control Instructions

Basic Data Transfer Instructions of 8085

Move

MOV Rd, Rs

Move immediate

MVI R, 8 bit data

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Basic Data Transfer Instructions of 8085

Input from port

IN 8-bit address of the input port

Output to port

OUT 8-bit address of the output port

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Basic Arithmetic Instructions of 8085

- Add register to Accumulator

ADD R

- Add immediate to Accumulator

ADI 8-bit data

- Subtract register from Accumulator

SUB R

- Subtract immediate from Accumulator

SUI 8 bit data

Basic Arithmetic Instructions of 8085

Increment register

INR R

Decrement register

DCR R

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Basic Logical Instructions of 8085

- AND with Accumulator

ANA R

- AND Immediate with Accumulator

ANI 8-bit data

- OR with Accumulator

ORA R

- OR Immediate with Accumulator

ORI 8-bit data

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Basic Logical Instructions of 8085

- Exclusive-OR with Accumulator

XRA R

- Exclusive-OR Immediate with Accumulator

XRI 8-bit data

- Complement Accumulator

CMA

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Basic Branching Instructions of 8085

- Unconditional Jump

  JMP Address (Label)

- Conditional Jump Instructions

  Jump on zero

  JZ Address (Label)

  Jump on non-zero

  JNZ Address (Label)

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Basic Branching Instructions of 8085

- Jump on plus

  JP Address (Label)
- Jump on minus

  JM Address (Label)
- Jump on even parity

  JPE Address (Label)
- Jump on odd parity

  JPO Address (Label)

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Machine Control Instructions of 8085

- Halt

HLT

- No Operation

NOP

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Reference

- Microprocessor Architecture, Programming, and Applications with the 8085, by Ramesh S Gaonkar, Fifth Edition, Prentice Hall

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Introduction to GNUSim8085

- GNUSim8085 is a graphical simulator, assembler and debugger for the Intel 8085 microprocessor.
- It is an open-source simulation tool.
- Ease the users to code, assemble and execute 8085 assembly programs without using actual hardware.
- Provides a user-friendly interface.

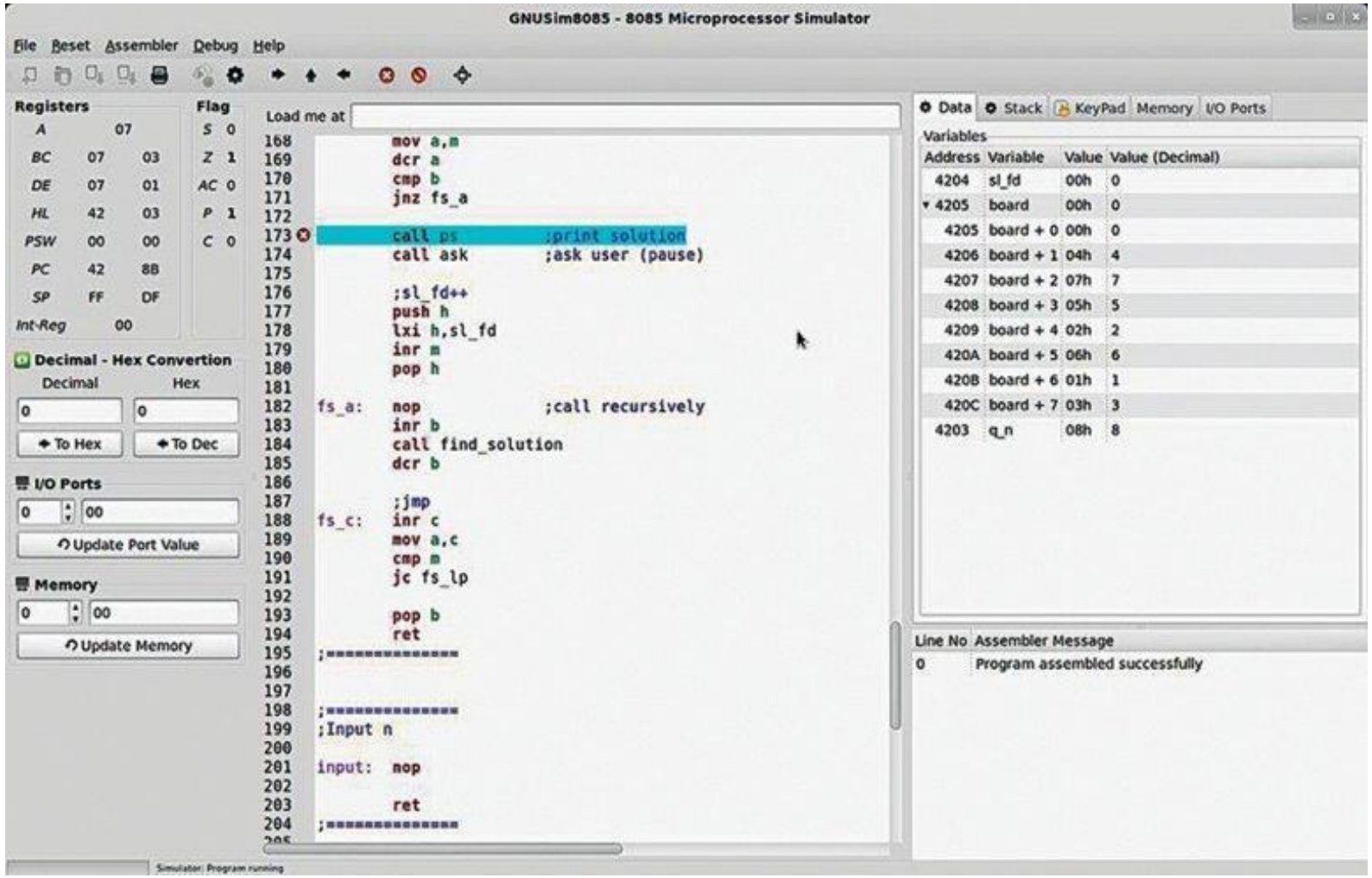
Features of GNUSim8085

- Clear depiction of registers, ports and memory in its GUI.
- The simulation has input and output mechanism identical to the actual hardware.
- User can also view input/output (I/O) ports, memory and stack contents.
- The software can display the status of processor's registers and flags.

Advantage:

- Introduction of an assembler and an editor for the 8085 processor.
- The editor comes with syntax highlighting.

Using the Software



Using the Software

- The software launches a workspace with sample codes that can be worked on.
- Users can open a new project from File->New.
- A toolbar on the top contains various options including File, Reset, Assembler, Debug and Help.
- The user interface (UI) is very comprehensive and user-friendly.
- The left panel contains a compact overview of the Intel processor's registers and flags.

Using the Software

- The hexadecimal-to-decimal converter is located in this panel as well.
- The value to be converted needs to be inserted manually.
- Below the converter, two spin-boxes are available.
- One is for the I/O port value and the other for the memory register.
- Entering the port number in the numeric box displays the port content in the adjacent text box.

Using the Software

- Users can update port values from these spin-boxes.
- The right panel comes with multiple tabs that list values of the processor's different entities.
- The first tab is 'Data,' which enlists the defined data variables.
- The following tab is 'Stack,' which displays addresses and values of stack region of the program.
- Two other tabs include I/O port and memory port, which enlist the values of I/O ports and memory.

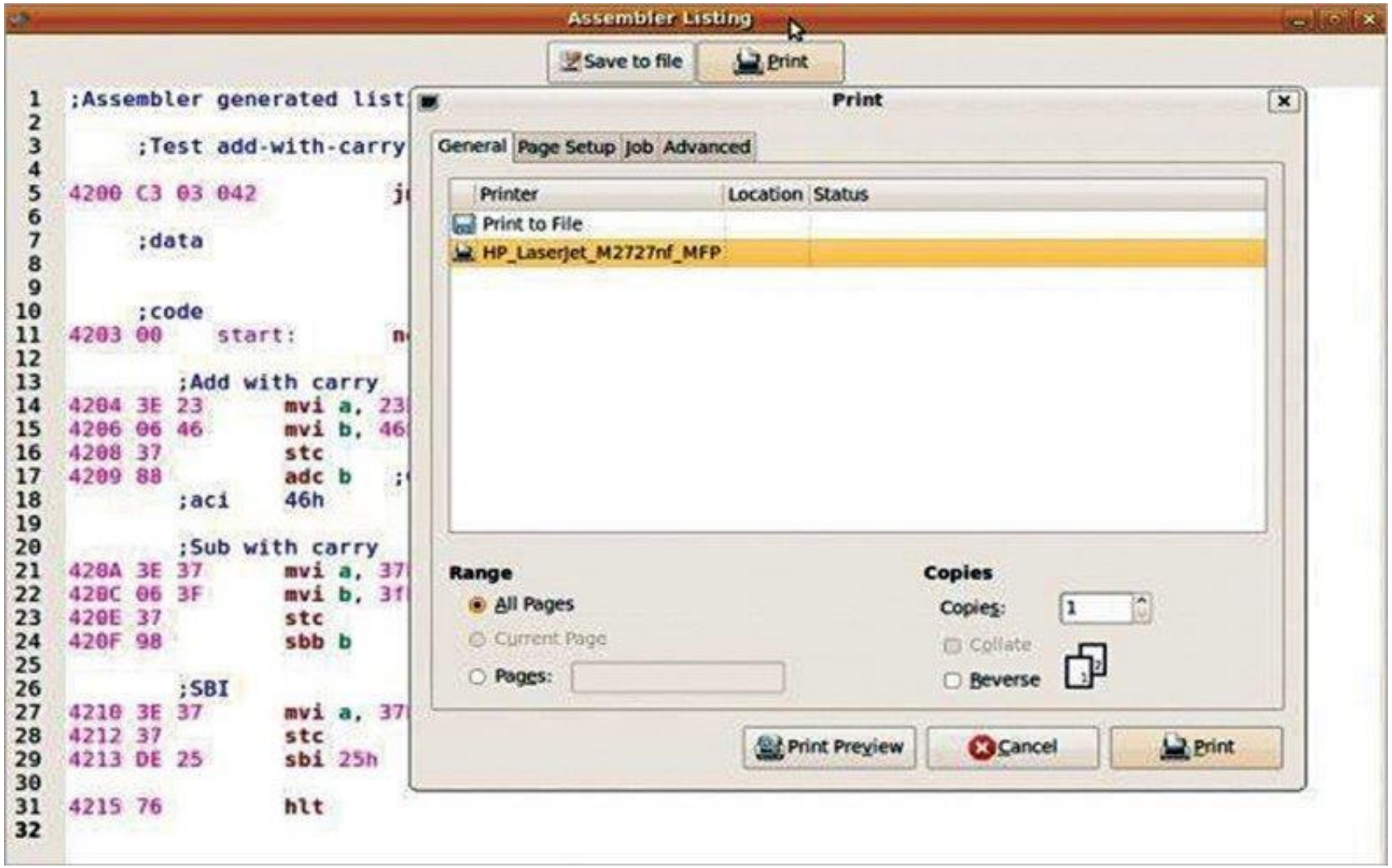
Using the Software

- An additional tab opens a virtual keypad, which allows users to digitally insert values.
- At the lower end of the right panel, a message box that displays the status of program compilation and execution.
- If any error, the message box displays an error message mentioning the location and reason of error.
- Successful program execution displays a success message.

Using the Software

- "Assembler" is an option available in the top menu of the software.
- This will convert the code written in mnemonics to 8085-compatible machine code.
- The first is "Assemble", which loads the program code to the memory address.
- The second is "Execute", which compiles and runs the loaded program.
- The third option is "Show Listing", where users can view the executed program code.

Using the Software



Using the Software

- The simulator toolbar has two other menu options.
- One is "Debug" which consists of sub-menus "Step In", "Step Over" and "Step Out".
- Debug menu also comes with a code breakpoint feature.
- This makes code debugging easy by analyzing the register and memory content in each step.

Using the Software

- Reset menu will reset the values of registers, ports, flags and memory.
- It help users to erase old data from the simulator.
- It can be done individually for each component or for all at once.

Sample 8085 Programs

Program:

Following program adds two 8bit values stored in registers A and B and save the result in A.

```
MVI   A, 05
MVI   B, 02
ADD   B
HLT
```

Explanation:

- MVI A, 05 - will load the value 05 to Accumulator.
- MVI B, 02 - will load the value 02 to register B.
- ADD B - will add the content of register B with A and save the result in A by default.
- HLT - will indicate the end of the program